

Aggregation and message passing: standard pool, heuristic, and variational server

Aggregation and message... (part 2)

The server step lives in `aggregation.py`, where a categorical specialization of the active-inference belief-sharing relation (Friston et al. 2024) and the FedGVI objective (Mildner et al. 2025) meet. Each agent n broadcasts a categorical local posterior $q_n(s)$ over the shared latent factor,

Aggregation and message... (part 3)

optionally with a scalar base weight w_n .

Two fusion rules act directly on these broadcasts, and a third — the objective-backed `variational_aggregate` of sec. 5.8.2 — refines the second into descent on a stated objective.

Aggregation and message... (part 4)

The first is the **log-linear pool**, a project-local product-of-experts consensus. In the terminology of opinion pooling it is the logarithmic pool,

Aggregation and message... (part 5)

a weighted geometric aggregation rule whose Bayesian-coherence assumptions have been studied independently of active inference (Genest and Zidek 1986; Genest et al. 1986; Carvalho et al. 2023); in machine-learning terms it is the product-of-experts normalization of local posteriors (Hinton 2002):

Aggregation and message... (part 6)

$$\begin{aligned} \log_linear_pool(\{q_n\}) \\ = \text{softmax}\left(\sum_n w_n \log q_n\right). \end{aligned} \tag{9}$$

Aggregation and message... (part 7)

In the floating-point implementation, q_n denotes the admitted posterior after input validation, lower flooring of masses, and row normalization. The floor acts before normalization and applies to small positive masses as well as exact zeros. The pooling formula uses those admitted posteriors;

Aggregation and message... (part 8)

rescaling sufficiently small raw masses can change them.

For the source bridge, fix one finite shared support $\mathcal{S}$ with $q_n(s) > 0$ for every agent and state.

Aggregation and message... (part 9)

Suppose the inputs to Eq. 7's softmax message-combination term can be represented as posterior log potentials $m_n(s) = \log q_n(s) + \kappa_n$, where κ_n is constant in s , and use declared fixed weights w_n that do not depend on the emerging consensus (the unweighted case sets each $w_n = 1$).

Aggregation and message... (part 10)

Additive constants then cancel under softmax, giving exactly eq. 9.

Aggregation and message... (part 11)

This is a categorical posterior-log-potential specialization of the source equation's message-combination term, not a reconstruction of source message construction, self-exclusion/cavity policy, scheduling, generative factors, or the complete protocol.

Aggregation and message... (part 12)

The code alias `friston_belief_share` names this qualified specialization only.

Aggregation and message... (part 13)

The second rule is **robust_aggregate**, an iteratively-reweighted pool that discounts each agent by $\exp(-c \text{KL}(q_n \parallel q))$ against the emerging consensus q . A confidently-wrong (contaminated) agent sits far from the consensus,

Aggregation and message... (part 14)

can earn a small effective weight and be suppressed in the declared diagnostic regimes.

Aggregation and message... (part 15)

This independently motivated rule does not transfer FedGVI's client theorem to the server side: it is the **heuristic robustness axis** of sec. 3.3, distinct from the per-agent rigorous axis of sec. 5.7.

Aggregation and message... (part 16)

It is also only an analogy to robust federated aggregation methods such as divergence-weighted gamma-mean aggregation, geometric-median robust aggregation, or Byzantine-tolerant gradient aggregation (Li et al. 2022; Pillutla et al. 2022; Blanchard et al. 2017).

Aggregation and message... (part 17)

Those methods motivate the risk surface, but they do not supply this rule's guarantee.

The defining identity is bit-level: at zero robustness the reweighted pool is the log-linear pool unchanged.

Aggregation and message... (part 18)

$$\text{robust_aggregate}(0) \equiv \text{log_linear_pool}. \quad (10)$$

Aggregation and message... (part 19)

This is an exact project-local code identity. Under the stated posterior-log-potential assumptions, its right-hand side specializes the message-combination term of Eq. 7; the identity itself neither recovers nor certifies the complete source protocol (Friston et al. 2024).

Protocol map: local updates, broadcast, and server fusion

The visual map in fig. 3 makes the protocol boundary explicit: each client updates and broadcasts a categorical posterior; the server chooses the standard pool, heuristic, or variational route.

Protocol map (part 2)

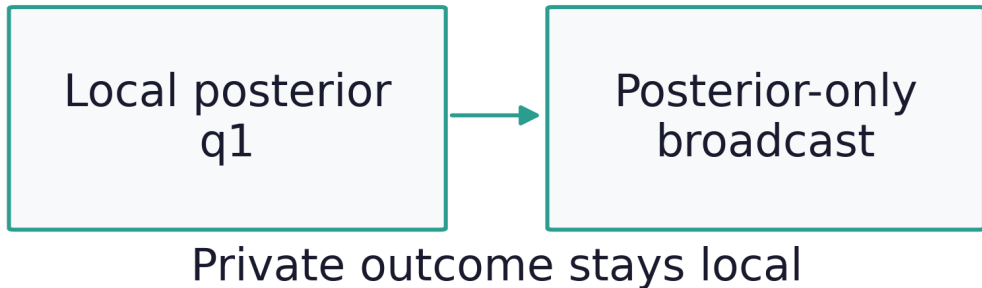
This is a mechanistic schematic, not an additional benchmark: client-side FedGVI is source-conditional, server-heuristic accuracy is conditional on declared contamination, and the variational route owns objective/descent/raw-weight properties.

Protocol map (part 3)

Private update: local q1

1	2	3
4	5	6
7	8	9

Protocol map (part 4)

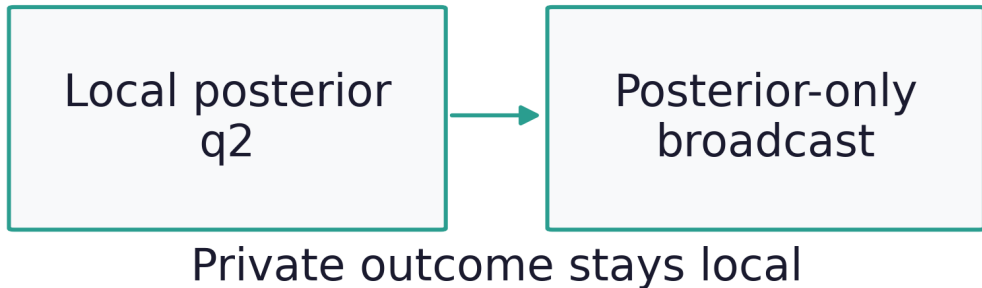


Protocol map (part 5)

Private update: local q2

1	2	3
4	5	6
7	8	9

Protocol map (part 6)

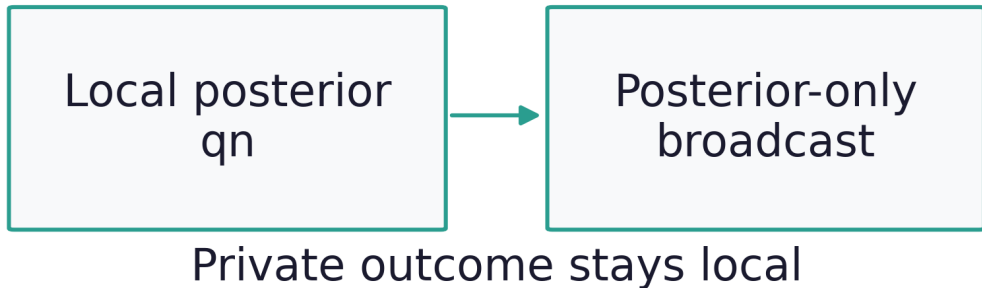


Protocol map (part 7)

Private update: local qn

1	2	3
4	5	6
7	8	9

Protocol map (part 8)



Protocol map (part 9)

Client route: $NLL/KLD/\beta = 0$ is a project recovery identity.

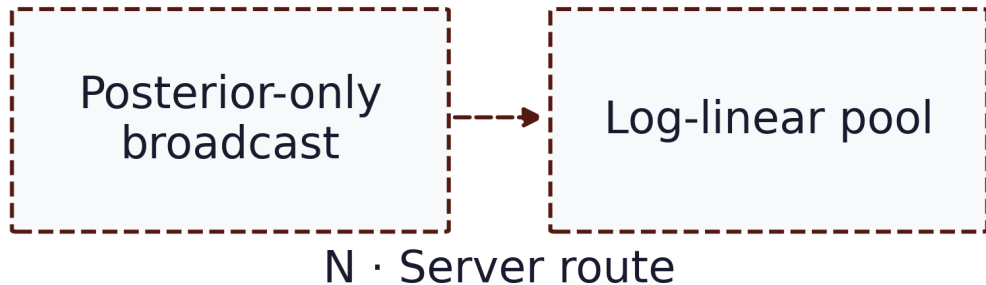
Protocol map (part 10)

Robust client losses: source theorem under its assumptions. Client guarantees do not transfer to server routes.

Protocol map (part 11)

Protocol-v1 payload: categorical posterior $q(n)$. No private outcome $o(n)$ is broadcast.

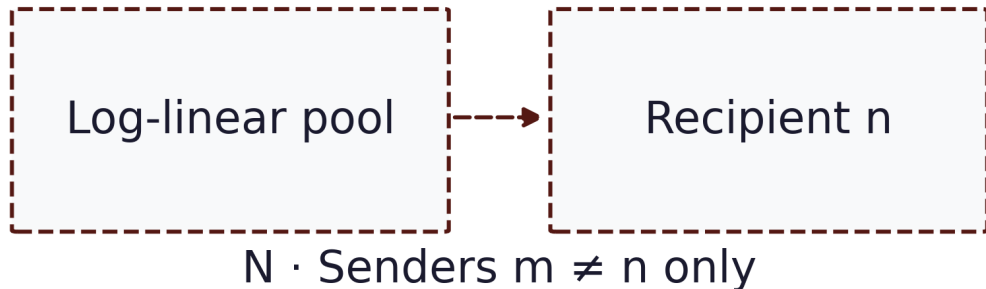
Protocol map (part 12)



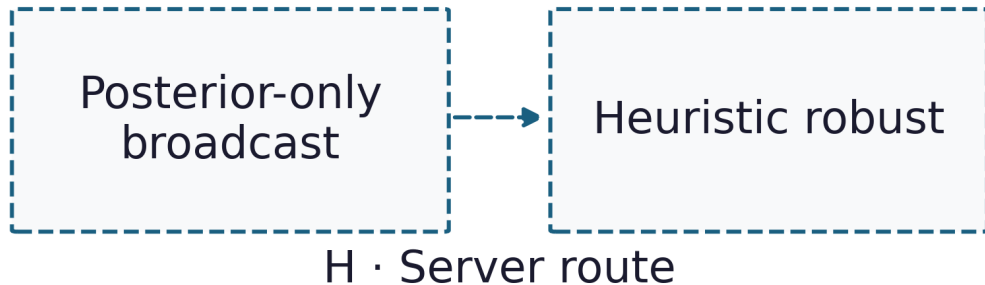
Protocol map (part 13)

N: softmax of the weighted sum of log posteriors. Qualified categorical Eq. 7 specialization.

Protocol map (part 14)



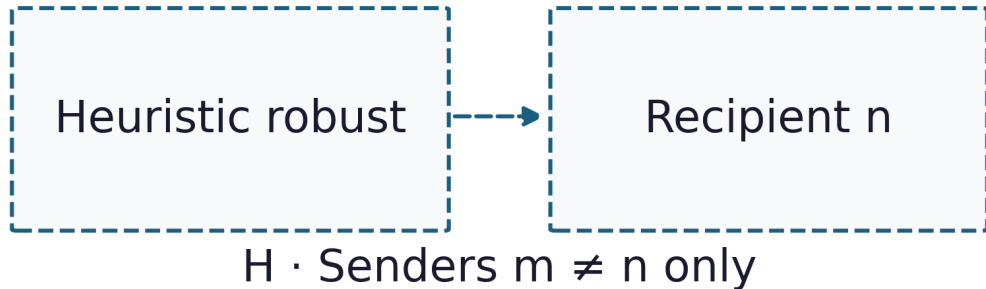
Protocol map (part 15)



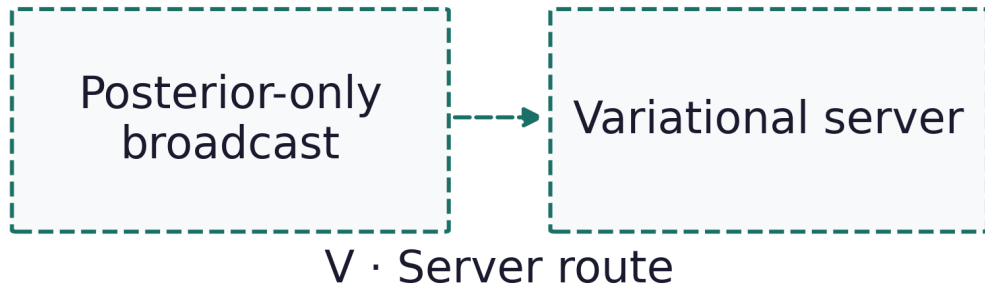
Protocol map (part 16)

H: robust_aggregate uses configured divergence reweighting. Conditional heuristic evidence.

Protocol map (part 17)



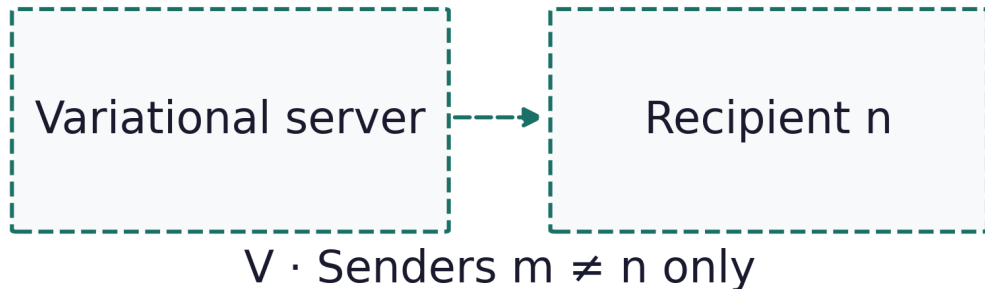
Protocol map (part 18)



Protocol map (part 19)

V: variational_aggregate uses q, a
block updates of $F(q, a)$.
Objective-backed weight property.

Protocol map (part 20)



Protocol map (part 21)

Deterministic protocol schematic; no empirical estimate or uncertainty interval. Self-exclusion is preserved. The Eq. 7 bridge does not reconstruct the full source protocol.

Protocol map (part 22)

Theorem (Categorical combination and recovery)

For finite $\mathcal{S}$ and $q_n(s) > 0$, take Eq. 7 inputs $m_n(s) = \log q_n(s) + \kappa_n$, with state-constant κ_n and fixed w_n . $\text{softmax}(\sum_n w_n m_n)$ equals (9). At $c = 0$, the server returns it by (10). This identifies categorical combination and recovery only, not the protocol or $c > 0$ behavior.

Protocol map (part 23)

At $c = 0$, every reweighting multiplier is $\exp(0) = 1$, the iteration is skipped, and the same pool code path returns.

Protocol map (part 24)

Corollary (Closed-form Bayes recovery)

With KL/NLL , the posterior of (2) is the prior-times-likelihood product in (11). Thus $generalized_posterior(KLD, NLL)$ reproduces Bayes. Under Theorem 5, pooling those posteriors specializes Eq. 7's categorical combination term, not its complete protocol.

Protocol map (part 25)

$$q^*(s) \propto \pi_0(s) \prod_i p(o_i | s), \quad (11)$$

Protocol map (part 26)

Pooling gives the project log-linear pool of eq. 9.

Protocol map (part 27)

The largest observed discrepancy between `robust_aggregate(robustness=0)` and `log_linear_pool` is 0 — bit-identical, since the zero-robustness branch runs the same code path — and between `generalized_posterior(KLD, NLL)` and the analytic Bayes posterior is $5.55\text{e-}17$,

Protocol map (part 28)

exact to machine precision (about one ULP); both are reported in sec. 7.1, so eq. 10 and eq. 11 are verified identities rather than approximations.

Protocol map (part 29)

The honesty contract binds at exactly this point. The recovery theorem and its corollary cover only the recovery identity and the per-agent rigorous axis of sec. 5.7; no statement *about* *robust_aggregate* transfers a bounded-influence guarantee to that divergence-reweighting,

Protocol map (part 30)

whose positive property is the robustness $= 0$
limit of eq. 10.

Protocol map (part 31)

A scoped no-go rejects a declared separable objective class without supplying a broader objective certificate. The per-agent influence weights that the heuristic produces under contamination are *illustrated*, not guaranteed, in fig. 15.

Protocol map (part 32)

The separate fig. 18 is an exploratory generalized-Bayes point-estimate logistic-regression baseline comparing joint NLL/L2 and RCCE/L2 configurations. Because both loss and shrinkage differ, it does not isolate an RCCE-only effect or test the genuine per-client FedGVI property,

Protocol map (part 33)

which remains source-conditional in sec. 5.7.

The next subsection closes this exact gap on the server side with a *different*, objective-backed aggregator.

Variational aggregation with objective-backed weight control

The related server construction becomes a genuinely variational rule by replacing the heuristic's reverse-KL weight update with a forward cross-entropy update. For $c > 0$ and $\lambda > 0$,

Variational aggregation with... (part 2)

treat the consensus q and a vector of effective weights $a = (a_n)$ as joint variational parameters and define the **aggregation free energy**

Variational aggregation with... (part 3)

$$F_{\lambda}(q, a) = \sum_n a_n \text{CE}(q, q_n) - \lambda H(q) + \frac{1}{c} \text{KL}_{\text{gen}}(a \parallel w). \quad (12)$$

Variational aggregation with... (part 4)

where $\text{CE}(q, q_n) = -\sum_i q_i \log q_{n,i}$ is the cross-entropy of the consensus relative to agent n , $H(q)$ is the consensus entropy, c is the robustness, and $\lambda > 0$ is the entropy_weight coefficient (default $\lambda = 1.0$);

Variational aggregation with... (part 5)

$\text{KL}_{\text{gen}}(a \parallel w) = \sum_n [a_n \log(a_n/w_n) - a_n + w_n]$
is the generalized KL between the effective and base weights.

Each block of F_λ has a closed-form minimizer, so alternating

Variational aggregation with... (part 6)

$$\begin{aligned} q &\leftarrow \text{softmax}\left(\frac{1}{\lambda} \sum_n a_n \log q_n\right), \\ a_n &\leftarrow w_n \exp(-c \text{CE}(q, q_n)). \end{aligned} \tag{13}$$

Variational aggregation with... (part 7)

is exact block-coordinate descent on eq. 12
(`variational_aggregate`) for $c > 0$ and
 $\lambda > 0$.

Variational aggregation with... (part 8)

The implementation defines the $\lambda \downarrow 0$ endpoint separately as a deterministic tied-argmax rule; $\lambda = 0$ is not substituted into the objective or its q -update.

Variational aggregation with... (part 9)

This substitution changes both orientation and scale: $\text{CE}(q, q_n) = \text{KL}(q \| q_n) + H(q)$, and its common $H(q)$ term scales all raw weights, which changes the entropy of the subsequent unnormalized weighted log pool.

Variational aggregation with... (part 10)

The paired q - and a -updates in eq. 13 are exact block minimizers of the stated objective; that fact does not derive the reverse-KL heuristic.

Variational aggregation with... (part 11)

Because F is biconvex, we run the descent **multi-start** (pool, uniform, and arithmetic-mean seeds, lowest observed F among converged starts;

Variational aggregation with... (part 12)

otherwise the lowest unfinished trace is returned with `converged=False`) so a near-one-hot adversary is not left at the product-of-experts seed in the tested contamination regimes — the detail that supports the effective-weight diagnostic (sec. 12).

Variational aggregation with... (part 13)

The full derivation, the formal statement (block descent, $c \rightarrow 0$ recovery, and the raw effective-weight bound), and the numerical witnesses are in sec. 12;

Variational aggregation with... (part 14)

the empirical descent and influence collapse are shown in fig. 16 and fig. 17 and reported in sec. 7.5.3.

Variational aggregation with... (part 15)

The authoritative notation supplement records the complete objective contract in eq. 34.

Variational aggregation with... (part 16)

This upgrades the server side from an untracked heuristic to a derived generalized-Bayes aggregation with an explicit redescending raw-weight property: a single confidently-wrong agent earns raw weight

$a_n = w_n \exp(-c \text{CE}(q, q_n)) \leq w_n$ that vanishes as it diverges,

Variational aggregation with... (part 17)

whereas the naive pool grants every agent the fixed weight w_n however wrong it is.

Variational aggregation with... (part 18)

The trade is conservatism — the $-H(q)$ term makes the stationary point a maximum-entropy-biased consensus consistent with the weighted cross-entropies, so `variational_aggregate` is deliberately flatter than the product-of-experts and does *not* maximize peak point-accuracy.

Variational aggregation with... (part 19)

The two server-side rules therefore play complementary, never-conflated roles, both reported:

Variational aggregation... (part 20)

the sharp `robust_aggregate` heuristic for accuracy under contamination (sec. 7.5.2) and the conservative `variational_aggregate` for a server-side objective with stated weight control (sec. 7.5.3).

Variational aggregation with... (part 21)

A temperature parameter $\lambda > 0$ (controlled by `entropy_weight`, default $\lambda = 1.0$) generalizes the objective to F_λ ; lower λ sharpens the variational q -block toward a maximizing state for its current weighted log pool.

Variational aggregation... (part 22)

The tempered family (sec. 12.5; objective eq. 30) recovers the full-entropy variational aggregator at $\lambda = 1.0$ and has a separately implemented deterministic tied-argmax endpoint as $\lambda \downarrow 0$;

Variational aggregation... (part 23)

neither endpoint is guaranteed accurate and
neither is an algebraic recovery of
`robust_aggregate`.

The effective-weight a -update is unchanged for
every $\lambda > 0$, so the raw-weight bound holds over
the objective-defined family.